Project 3
CSE 464
Gautam Mehta

1226288195

Project Description
Continuing Project 1 and adding 3 new APIs to remove node/nodes and edge and their test cases.

Parse a DOT file to create a directed graph object.

- Add nodes and edges to the graph.
- Output the graph details as text.
- Export the graph in DOT format and generate a graphical output (e.g., PNG) using Graphviz.

Prerequisites

- Java JDK 11 or above
- Maven 3.6+
- Graphviz installed and available in your system PATH
- Git

Intstructions to run:

Clone this repo https://github.com/skidiKidiKaka/CSE-464-2025-gmehta10.git

Navigate to this project directory : CSE-464-2025-gmehta10

Run this command to compile and test : mvn clean package

Then go to the main file and run it using the run button in Intellij app

Refactor 1 Extract Method (normalizeLine) GraphParser.java:
I extract trim-and-semicolon-removal code from parseGraph into new helper normalizeLine. Now parseGraph is more simple and less duplicate, so code become easier to read and maintain.

https://github.com/skidiKidiKaka/CSE-464-2025-gmehta10/commit/837e61dfc8139bc6d4d743a8d20e3274ca55ebe3

Refactor 2: Extract Variable neighbor in BFS/DFS methods:
I pull out edge[1] into local variable to (neighbor) inside the bfs/dfs loops. this reduce repeating the array lookup and make the search logic more clear and easy to read.

https://github.com/skidiKidiKaka/CSE-464-2025-gmehta10/commit/f48fec31a020dc47c46604e19312909eb8490c97

Refactor 3: Extract Method – writeToFile in GraphParser.java
I add new helper writeToFile to do all file-opening, writing and exception handling. then both outputDOTGraph and outputGraph just call it, so no duplicate try/catch code and easier to change.

https://github.com/skidiKidiKaka/CSE-464-2025-gmehta10/commit/1de2fe69f0a3fedcc32dbd2502f721c037673111

Refactor 4: Rename Method – normalizeLine to cleanLine
I rename helper method from normalizeLine to cleanLine so name more match its job of cleaning input line. this make code more clear and intention more obvious.

https://github.com/skidiKidiKaka/CSE-464-2025-gmehta10/commit/aa31acb64076af6ad68f6c130a3e7c383ba66a89

Refactor 5: Extract Method – appendEdges in toDOTString
I pull out the repeated for-loop that writes each edge[0] -> edge[1]; line into new helper appendEdges, so toDOTString become shorter and easier to read.

https://github.com/skidiKidiKaka/CSE-464-2025-gmehta10/commit/8054708f0b4c0f7d5e3ae63b605ed028893ae423

Question 2: Template Pattern Design Description

I created new files GraphSearchTemplate.java, BFSTemplate.java, and DFSTemplate.java to abstract BFS and DFS common logic. in GraphSearchTemplate put shared steps like init frontier, loop search, build path. then BFSTemplate use queue for FIFO, DFSTemplate use stack for LIFO. update GraphParser.java to call GraphSearch(..., algo) which dispatch to these templates. this way code duplicate gone and easy add new search algo.

https://github.com/skidiKidiKaka/CSE-464-2025-gmehta10/commit/1bd3b6aeff7b7e5dcf8ee1e47a0abc5c62a82de9

Question 3: Strategy Pattern Design Description

I added new interface GraphSearchStrategy and factory GraphSearchStrategyFactory to pick BFS or DFS at runtime. BFSTemplate and DFSTemplate both implement this interface (and reuse our template-method logic), so parser just calls factory.getStrategy(algo) and then search(start,target). this way code no longer hard-coded, can switch algorithm by passing Algorithm.BFS or Algorithm.DFS to the same GraphSearch(...) API.

https://github.com/skidiKidiKaka/CSE-464-2025-gmehta10/commit/b4d066f9824bdb9d26e516d1533ae89c4420e3e8#diff-a71900caf323be0ceb8ece8edaaba28584cac7a843087031624a291a042bf656

Question 4: Random Walk Search

I extended our Algorithm enum with a new RANDOMWALK option and wrote a RandomWalkTemplate class that keeps its frontier in a List and picks a node at random each step. Then I plugged it into GraphParser.search(…,Algorithm) alongside BFS and DFS, so the same enum dispatch now handles random-walk too. Finally, in Main I ran several searches from a to c to show how each run produces a different path, proving that adding a new strategy with our template setup is really straightforward.

https://github.com/skidiKidiKaka/CSE-464-2025-gmehta10/commit/94b4ae0827be288925a85e794e0be6bbe5122479

Pull Request Link:
https://github.com/skidiKidiKaka/CSE-464-2025-gmehta10/pull/3